

Relatório atividade prática 2

Iago Zagnoli Albergaria
Universidade Federal de Minas Gerais
Belo Horizonte, Brasil
iagozag@ufmg.br

ACM Reference Format:

Iago Zagnoli Albergaria. 2026. Relatório atividade prática 2. In . ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Estruturas de dados

Nesta seção são descritas as estruturas de dados utilizadas no processador de consultas e no indexador. Além disso, é explicada a complexidade de cada uma das estruturas.

1.1 Indexer

A indexação deste trabalho utiliza diversas estruturas de dados para garantir a eficiência no processamento e armazenamento das informações. A principal estrutura utilizada é o índice invertido, implementado por meio de um `defaultdict(list)` na classe `Indexer`. Nessa estrutura, cada termo do vocabulário funciona como chave de um dicionário e está associado a uma lista de *postings*. Cada *posting* é representado por uma tupla contendo o identificador do documento e a frequência do termo naquele documento. Dessa forma, um termo é associado a uma lista de tuplas, indicando em quais documentos o termo aparece e quantas vezes ocorre em cada um deles.

O `defaultdict` é utilizado para o gerenciamento das estruturas internas do índice. Cada novo termo recebe uma lista vazia como valor inicial. Além disso, a estrutura também é usada para armazenar frequências de termos em documentos individuais, nos quais cada termo é associado a um contador inteiro.

As listas neste trabalho são utilizadas para armazenar *postings*, conjuntos de documentos e arquivos temporários. As listas de *postings* armazenam pares, enquanto *batches* são utilizados para agrupar documentos antes do processamento paralelo. Em determinados momentos do trabalho, essas listas precisam ser ordenadas, especialmente antes da gravação dos *postings* em disco.

A implementação também utiliza *heaps*, que implementa uma fila de prioridade baseada em *heap* mínimo. Essa estrutura é utilizada na etapa de *merge* externo dos arquivos temporários gerados durante a indexação. Como este projeto lida com grandes volumes de dados e memória limitada, o índice invertido é periodicamente descarregado para arquivos parciais no disco. Posteriormente, esses arquivos são reunidos utilizando *merge* baseado em *heap*.

A classe `Indexer` centraliza todas as estruturas e informações relacionadas ao processo de indexação. Ela atua como o centro do indexador, mantendo em memória o índice invertido parcial, os arquivos temporários, o conjunto de *stopwords*, os caminhos dos arquivos e as configurações de memória disponíveis. Além disso, a classe encapsula todos os componentes necessários para transformar o corpus textual em uma estrutura eficiente de recuperação da informação.

Por fim, a implementação utiliza arquivos JSON e TSV como estruturas persistentes de armazenamento. Os arquivos TSV são usados para armazenar temporariamente dados intermediários durante o processo de indexação, enquanto os arquivos JSON armazenam o léxico e as estatísticas finais dos documentos. Essa estratégia permite reduzir o consumo de memória principal e facilita o processamento de coleções muito grandes.

1.2 Processor

No processador de consultas são utilizadas diversas estruturas de dados para realizar a recuperação de modo eficiente a partir do índice invertido previamente construído. A classe `QueryProcessor` é responsável por carregar as estruturas auxiliares do índice, pré-processar as consultas e executar o algoritmo de recuperação baseado em *Document-at-a-Time*, conforme especificado na descrição do projeto. O objetivo dessa classe é realizar a busca de forma eficiente.

Uma das estruturas utilizadas no processador é o dicionário (*dict*), empregado em diferentes partes do processador. No léxico carregado, cada termo é associado a informações importantes para a recuperação da informação, como o deslocamento da lista de *postings* dentro do arquivo e a frequência no documento. O sistema também utiliza outro dicionário responsável por armazenar o tamanho de cada documento da coleção. Cada identificador de documento funciona como chave e o tamanho correspondente é armazenado como valor. Além disso, temos um *set* para armazenar as *stopwords*. Dessa forma, durante o pré-processamento da consulta, cada token pode ser rapidamente testado para verificar se deve ou não ser removido do processamento. Após a tokenização e *stemming* da consulta, os termos resultantes são adicionados em conjunto para eliminar duplicatas.

As listas armazenam os termos processados da consulta, as listas de *postings* recuperadas do índice invertido e os resultados finais das buscas. As listas de *postings* contêm pares no formato (*doc_id*, *tf*), em que *doc_id* é o identificador do documento e *tf* corresponde à frequência do termo naquele documento.

Durante a execução do algoritmo DAAT, a implementação utiliza dicionários auxiliares. Um dos dicionários armazena em memória todas as listas invertidas necessárias para responder à consulta atual. Outro dicionário associa cada termo ao seu valor de frequência documental. Além disso, há outro que é utilizado para controlar os ponteiros de navegação dentro das listas de *postings*. Cada termo

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

possui um ponteiro associado que indica a posição atual de leitura em sua respectiva lista invertida.

A classe `QueryProcessor` centraliza todas as estruturas necessárias para o processamento das consultas e para a recuperação eficiente da informação. Ela mantém em memória o léxico, as estatísticas globais da coleção e os tamanhos dos documentos, permitindo acesso rápido durante a execução das buscas.

2 Algoritmos

Nesta seção são descritas os algoritmos no processador de consultas e no indexador. Além disso, é explicada a complexidade de cada um desses algoritmos.

2.1 Indexer

O processo de indexação realiza o algoritmo de parsing dos documentos, no qual cada linha do corpus é interpretada como um objeto JSON. Essa operação possui complexidade $O(n)$, onde n representa o tamanho da linha processada, já que é necessário percorrer todo o conteúdo textual para realizar a leitura dos dados. Após o parsing, executamos um algoritmo de normalização textual utilizando expressões regulares (*regex*). Esse algoritmo percorre toda a string caractere por caractere, apresentando complexidade linear $O(n)$, onde n corresponde ao tamanho do texto.

Em seguida, o projeto realiza tokenização utilizando a biblioteca NLTK. O algoritmo divide o texto em tokens. Como cada caractere do texto precisa ser analisado para identificar separadores e palavras válidas, a complexidade da tokenização também é $O(n)$.

Após a tokenização, o executa-se os algoritmos de remoção de stopwords e stemming. A remoção de stopwords ocorre através de verificações em um conjunto, cuja operação de busca possui complexidade média $O(1)$. Dessa forma, considerando t tokens no documento, o custo total da filtragem é $O(t)$. Em seguida, cada token válido é submetido ao algoritmo responsável por reduzir palavras às suas raízes morfológicas.

Outro algoritmo presente na implementação é o de construção do índice invertido em memória. Para cada termo encontrado, uma tupla no formato (`doc_id`, `tf`) é adicionada à lista de postings correspondente. Como a inserção ao final de listas possui custo $O(1)$, o custo total dessa etapa é proporcional ao número total de termos processados.

O trabalho utiliza processamento paralelo com `ThreadPoolExecutor` para acelerar o processamento textual. A leitura do *corpus* acontece de forma sequencial, mas os documentos são separados em *batches* de tamanho fixo. Cada lote é enviado para um conjunto de *threads*, responsáveis por executar ao mesmo tempo as etapas de *parsing*, tokenização, remoção de *stopwords* e *stemming*.

Quando o limite de memória é atingido, o programa executa um algoritmo de escrita parcial em disco. Antes da gravação, os termos do índice invertido são ordenados. A ordenação dos termos possui complexidade $O(v \log v)$, onde v representa o tamanho do vocabulário armazenado em memória naquele momento. Além disso, para cada termo, sua lista de postings também é ordenada pelo identificador do documento. Se uma lista contém p postings, a ordenação possui custo $O(p \log p)$.

Após a geração dos arquivos temporários, o sistema executa um algoritmo de merge externo. O merge externo utiliza uma estrutura

de heap mínimo para selecionar continuamente o menor elemento disponível entre os arquivos abertos. Se n representa o número total de linhas processadas e k representa o número de arquivos parciais, a complexidade do merge é $O(n \log k)$.

2.2 Processor

No processamento de consultas implementa-se os algoritmos responsáveis pelo processamento e recuperação de documentos a partir do índice invertido previamente construído. É realizado o pré-processamento textual das consultas, recuperação das listas de postings e cálculo de relevância utilizando funções de ranqueamento. Esses algoritmos foram desenvolvidos para minimizar acessos desnecessários ao disco e garantir eficiência na recuperação da informação.

O primeiro algoritmo executado pelo processador é o carregamento das estruturas auxiliares do índice. O parsing desses arquivos possui complexidade linear $O(n)$, onde n representa o tamanho total dos arquivos lidos, já que todo o conteúdo precisa ser lido da memória. Após o carregamento das estruturas, o sistema executa o algoritmo de pré-processamento das consultas.

O algoritmo de recuperação das listas de postings utiliza acesso direto ao arquivo de índice invertido por meio de deslocamentos em bytes armazenados no léxico. Para cada termo da consulta, uma busca é realizada no dicionário, cuja complexidade média é $O(1)$. Em seguida, o algoritmo posiciona o ponteiro do arquivo diretamente no início da lista invertida desejada. Essa operação possui custo $O(1)$, pois o deslocamento é conhecido previamente. Após isso, o realiza-se a leitura da linha correspondente no arquivo JSON da lista de postings, cuja complexidade é linear em relação ao tamanho da lista recuperada.

Após encontrar documentos candidatos, o sistema executa algoritmos de ranqueamento para calcular a relevância dos resultados. O primeiro algoritmo implementado é o TF-IDF, calculado com uma complexidade $O(1)$ para cada termo analisado. O segundo algoritmo de ranqueamento implementado é o BM25. Esse algoritmo considera a frequência do termo, o tamanho do documento e o tamanho médio da coleção para calcular a relevância de forma mais robusta. Assim como o TF-IDF, o custo do cálculo individual também é $O(1)$. O algoritmo de cálculo de score percorre todos os termos da consulta acumulando suas contribuições para cada documento recuperado. Se uma consulta contém t termos, a complexidade do cálculo do score para um documento é $O(t)$.

Após o cálculo das pontuações, os documentos recuperados são ordenados em ordem decrescente de relevância. A ordenação possui complexidade $O(r \log r)$, onde r representa a quantidade de documentos recuperados para a consulta. Após a ordenação, apenas os dez documentos mais relevantes são retornados ao usuário.

3 Análise

A Tabela 1 mostra que o processo de indexação foi executado utilizando um limite de memória de 1024 MB e processou um total de 4.641.784 documentos, sendo 206.137.073 tokens totais na coleção. Durante a construção do índice invertido, o programa gerou 13 arquivos parciais temporários devido ao mecanismo de controle de memória implementado no indexador. Sempre que o consumo

de memória ultrapassava o limite estabelecido, o índice parcial armazenado em memória era descarregado para o disco, permitindo que o sistema continuasse processando grandes volumes de dados sem exceder a memória RAM disponível.

Após o processamento completo da coleção, o algoritmo de merge externo era chamado para combinar os arquivos parciais em um único índice invertido final. O índice resultante apresentou tamanho total de 2351.78 MB, contendo 2.726.977 listas invertidas distintas. Além disso, o tamanho médio das listas de postings foi de aproximadamente 51.77 elementos por termo, indicando a distribuição média dos documentos associados a cada termo.

O tempo total de execução do processo de indexação foi de 2696.83 segundos, o que corresponde a aproximadamente 45 minutos. Considerando o volume de documentos processados e as operações realizadas, como parsing, tokenização, stemming, ordenação externa e merge baseado em heap, o desempenho obtido demonstra a eficiência da arquitetura implementada. O uso de processamento paralelo, estruturas de dados otimizadas e técnicas de indexação externa permitiu que o sistema realizasse a construção do índice de forma escalável mesmo sob restrição de memória.

Um ponto importante no desempenho do indexador foi a estratégia de paralelização utilizada. Apesar de a divisão dos documentos em *batches* e o uso de 5 *threads* terem trazido ganhos perceptíveis em relação a uma execução totalmente sequencial, os testes mostraram que aumentar ainda mais a quantidade de *threads* não reduzia necessariamente o tempo total de processamento, às vezes até aumentando. Isso acontece principalmente por dois motivos. O primeiro é a limitação do *Global Interpreter Lock* (GIL) do Python, que dificulta a execução realmente paralela de tarefas dependentes de CPU dentro do mesmo processo. O segundo é que a construção do índice depende bastante de operações de entrada e saída (I/O *bound*). Assim, depois de certo ponto, o gargalo (*bottleneck*) deixa de ser o processador e passa a ser a velocidade de leitura e escrita no disco, fazendo com que adicionar mais concorrência não traga ganhos relevantes.

Table 1: Estatísticas da construção do índice invertido

Métrica	Valor
Documentos processados	4.641.784
Total de Tokens	206.137.073
Arquivos parciais gerados	13
Tamanho do índice	2351.78 MB
Tempo total de execução	2696.83 s
Número de listas invertidas	2.726.977
Tamanho médio das listas	51.77
Memória disponível	1024 MB

As Tabelas 2 e 3 evidenciam o comportamento das duas estratégias de ranqueamento implementadas no sistema a partir de diferentes consultas textuais. Em ambas, a coluna Matches representa a quantidade de documentos recuperados pela busca conjuntiva, enquanto os demais valores descrevem a distribuição dos scores atribuídos aos documentos retornados.

Table 2: Caracterização dos resultados e distribuição de scores para TFIDF

Consulta	Matc.	Min.	Máx.	Méd.
physics nobel winners since 2000	0	-	-	-
christopher nolan movies	2	19.66	19.66	19.66
19th century female authors	10	24.09	27.28	25.15
german cs universities	0	-	-	-
radiohead albums	10	29.51	37.64	33.66

Table 3: Caracterização dos resultados e distribuição de scores para BM25

Consulta	Matc.	Min.	Máx.	Méd.
physics nobel winners since 2000	0	-	-	-
christopher nolan movies	2	17.06	19.92	18.49
19th century female authors	10	16.56	19.19	17.20
german cs universities	0	-	-	-
radiohead albums	10	20.43	22.47	21.22

Observa-se inicialmente que algumas consultas não produziram resultados, como *physics nobel winners since 2000* e *german cs universities*. Isso ocorre porque o modelo de recuperação implementado exige que todos os termos relevantes da consulta estejam presentes simultaneamente no documento para que haja correspondência. Além disso, não houve a relaxação retirando os números e palavras pequenas. Dessa forma, consultas mais específicas ou compostas por termos pouco frequentes tendem a reduzir a quantidade de documentos candidatos.

Entre as consultas que retornaram resultados, percebe-se que *radiohead albums* apresentou os maiores valores de score nos dois métodos de ranqueamento. No TFIDF, os documentos recuperados alcançaram valores entre 29.51 e 37.64, enquanto no BM25 os scores variaram entre 20.43 e 22.47. Esse comportamento indica uma forte associação entre os termos da consulta e os documentos encontrados, sugerindo alta relevância semântica dentro da coleção indexada.

A consulta *christopher nolan movies* retornou apenas dois documentos em ambos os métodos, evidenciando uma recuperação mais restrita. No entanto, os scores atribuídos aos documentos permaneceram elevados, indicando que os poucos documentos encontrados possuem alta relação com os termos pesquisados. Já a consulta *19th century female authors* apresentou maior quantidade de resultados, atingindo o limite de dez documentos retornados, além de uma distribuição de scores mais uniforme.

Comparando os modelos de ranqueamento, percebe-se que o TFIDF gera valores numéricos mais altos e com maior dispersão entre os documentos recuperados. O BM25, por outro lado, produz scores menores devido ao mecanismo de normalização baseado no tamanho dos documentos e no controle da frequência dos termos. Assim, embora ambos os métodos consigam identificar documentos relevantes, o BM25 tende a apresentar uma distribuição mais estável dos scores, enquanto o TFIDF destaca com mais ruído diferenças de relevância entre documentos.